Reabastecimento em voo com drones guiados por radar

Descrição do sistema, diagramas de blocos e resultados atuais

projeto swarm_intercept · outubro de 2026

O que é	Simulador e banco de testes para decidir qual drone descartável vai reabastecer qual aeronave, quando os drones só conhecem as aeronaves por pistas de radar ruidosas e cheias de falsos alarmes.
Pergunta	Redes neurais (imitação e reforço) alocam drones melhor que heurísticas clássicas (gulosas e algoritmo húngaro)?
Cenário	Relevo real Copernicus GLO-30 (São José dos Campos, 14 × 14 km), 1–4 aeronaves, 1–5 radares, bases de drones e zonas prioritárias.
Estado	Ambiente, heurísticas, 3 redes, avaliação paralela e replay prontos. As redes empatam com a heurística húngara no treino e ficam levemente abaixo nos cenários de teste.

1. Visão geral

O sistema tem três camadas: **definição de missões** (desenhadas à mão no construtor ou geradas em lote), **simulação** (um simulador físico 6-DOF para validação e um ambiente cinemático rápido para treinar e comparar decisores) e **análise** (métricas com carimbo de data e hora, pesos das redes e replays animados).

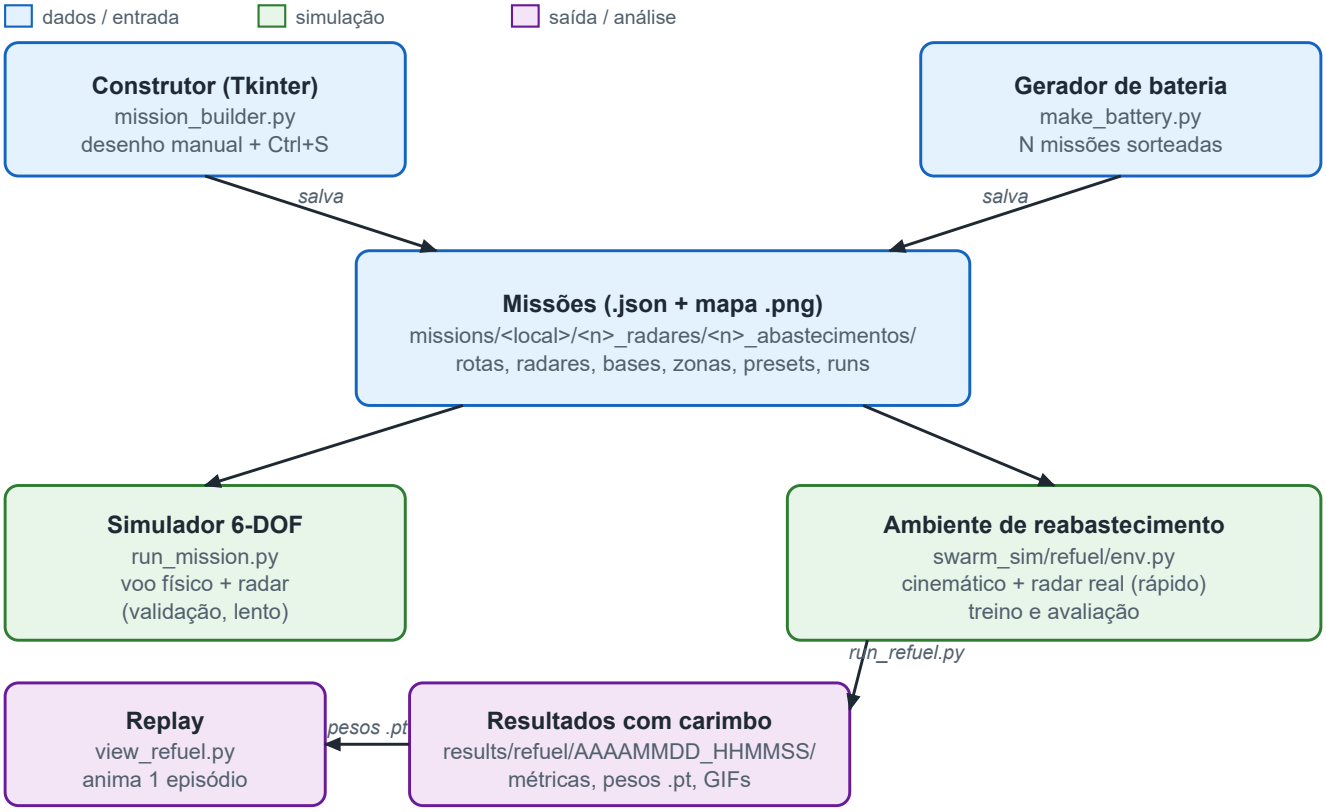


Figura 1 – Blocos do sistema e fluxo de arquivos.

1.1 O problema

Aeronaves voam rotas cujo último ponto é um **impacto no solo**: se não forem reabastecidas antes, caem ali. Drones partem de bases; cada drone reabastece **uma** aeronave e cada aeronave precisa de **um** drone. Uma queda dentro de uma **zona prioritária** custa 10 pontos; fora dela, 1. A dificuldade está na informação: os drones não sabem onde as aeronaves estão, só veem o **quadro de radar** (pistas com posição e velocidade estimadas, sem identidade), que só existe onde há cobertura e que, nas missões de São José dos Campos, tem cerca de **88 % de pistas falsas**.

2. A missão

Uma missão é um arquivo JSON salvo em `missions/<local>/<n>_radares/<n>_abastecimentos/`, sempre acompanhado de um mapa PNG gerado no momento em que é salva.

Elemento	Conteúdo
Mapa	Origem lat/lon e lado (ex.: 14 km). O relevo vem do DEM Copernicus de 30 m.
Aeronaves	Início, waypoints (altura, velocidade) e ponto final de impacto; RCS (seção radar).
Radares	Posição, altura da antena, alcance, ruído, vieses, setor, falsos alarmes por varredura.
Bases	Posição e número de drones.
Zonas prioritárias	Círculos; queda dentro custa 10, fora custa 1.
Drones (refuel)	Velocidade 42 m/s, autonomia 420 s, sensor próprio 300 m, contato 2 m.
Preset aleatório	Cada waypoint tem raio de sorteio, faixa de velocidade (22–32 m/s) e de altura (80–200 m); o início também é sorteado.
runs	Quantas execuções sorteadas a missão tem na avaliação; peso dela no sorteio do treino.

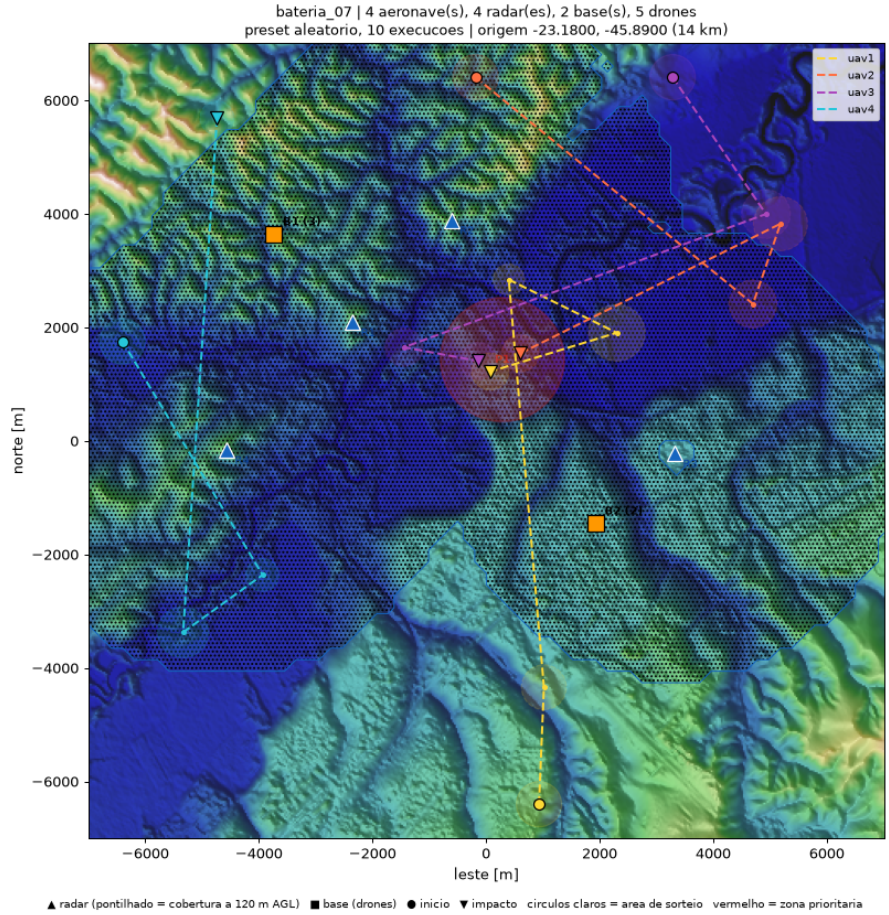


Figura 2 – Mapa salvo da missão bateria_07: 4 aeronaves (rotas tracejadas, círculos claros = área de sorteio, ▼ = impacto), 4 radares em morros com a cobertura a 120 m sobre o terreno (pontilhado), bases (■) e zona prioritária (vermelho).

2.1 Bateria de missões

`make_battery.py` gera missões em lote para um local. Cada uma combina 1–4 aeronaves e 3–5 radares; rotas saem da borda do mapa, passam por 2–3 pontos internos e terminam num impacto (60 % das vezes dentro de uma zona prioritária). Os radares são colocados nos morros que mais cobrem as rotas (cálculo de cobertura com relevo e difração); missões com rotas mal cobertas são refeitas. A bateria atual de São José dos Campos tem 16 missões, além de 3 desenhadas à mão.

3. O ambiente de reabastecimento

O ambiente (`swarm_sim/refuel/env.py`) é uma simulação de passo fixo de 1 s, simples o bastante para rodar milhares de episódios, mas com o radar completo. A Figura 3 mostra o caminho da informação: a posição verdadeira das aeronaves só alimenta os radares, o sensor de curto alcance dos drones e a decisão de contato e queda. O decisor nunca a vê.

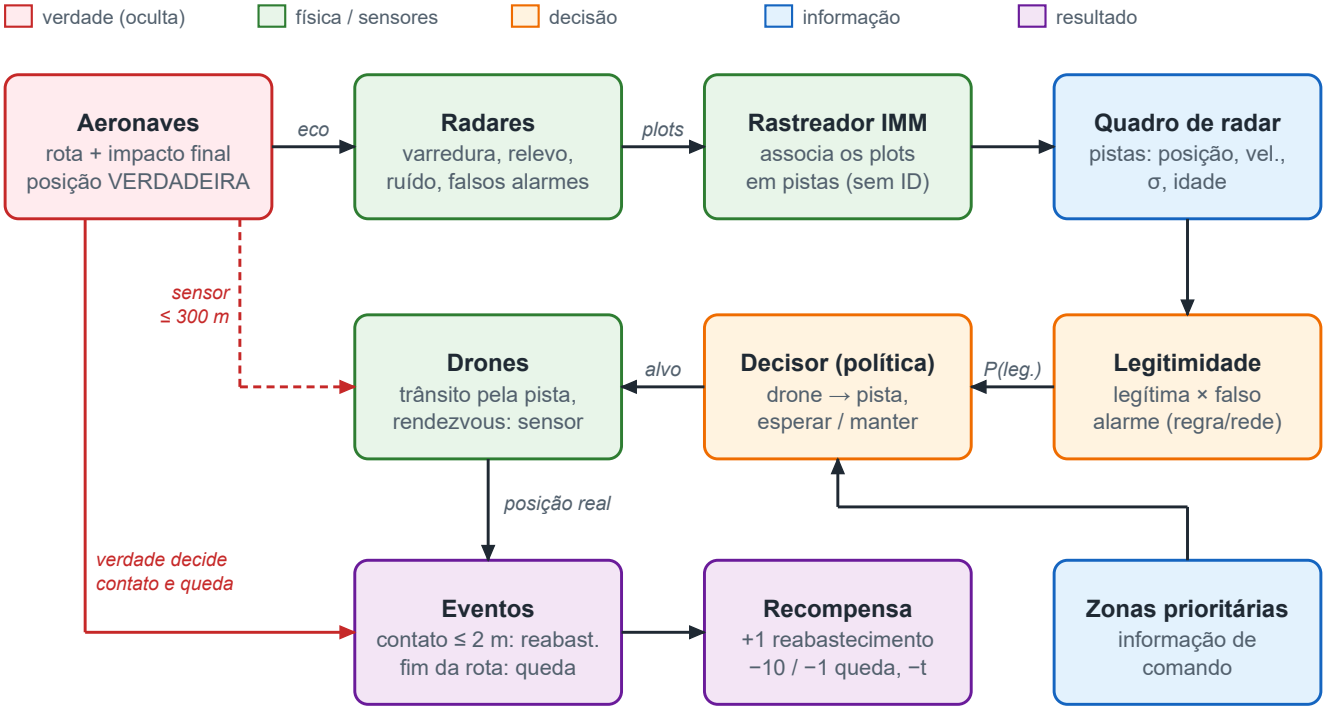


Figura 3 – Fluxo de informação no ambiente. Em vermelho, o que é verdade oculta para o decisor.

3.1 Radar e rastreamento

- O feixe gira; cada alvo é medido quando o feixe passa por ele, e só se houver linha de visada (relevo, difração em gume de faca, curvatura da Terra).
- Probabilidade de detecção pela equação do radar (alvo Swerling I, RCS dependente do aspecto); ruído de distância e azimute, vieses fixos e falsos alarmes concentrados perto do radar.
- Um rastreador IMM (dois modelos de velocidade constante) associa as medidas em **pistas** confirmadas: posição, velocidade, incerteza σ, idade, número de medidas. Sem identidade.

3.2 Drones: do lançamento ao contato

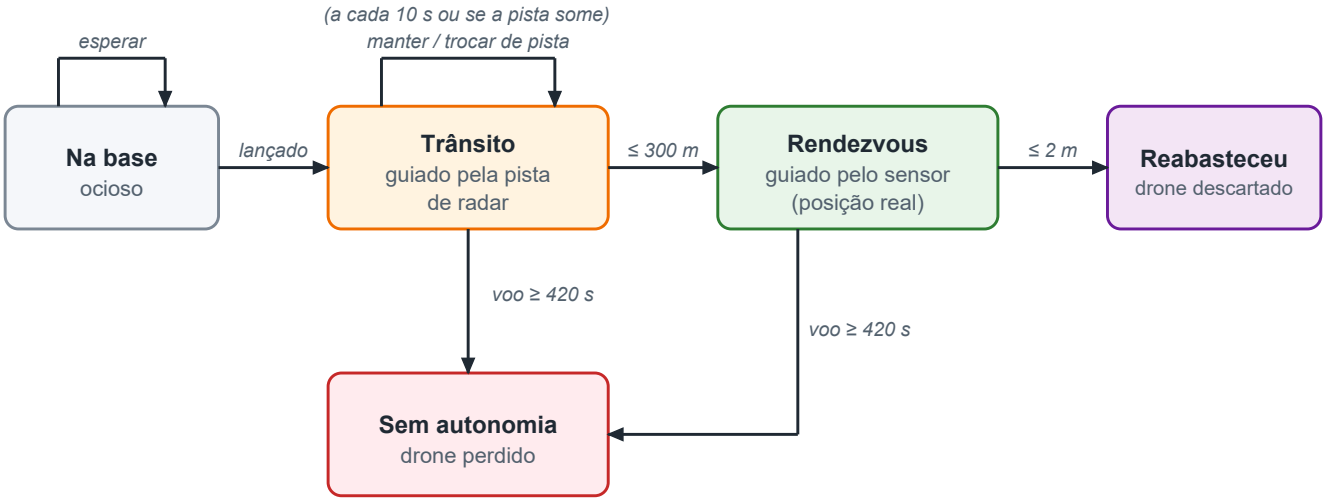


Figura 4 – Estados de um drone. Laranja: decisão do controlador; verde: guiagem automática pelo sensor próprio.

4. Como as decisões acontecem

A cada 2 s simulados há uma **rodada de decisão**. Entram na fila os drones em voo cuja reavaliação venceu (a cada 10 s, ou imediatamente quando a pista que perseguem some do radar) e, depois, os drones parados nas bases. A ordem importa: se um drone já no ar puder assumir uma pista, nenhum drone novo é gasto com ela. Cada drone da fila recebe sua observação e a política devolve uma ação.

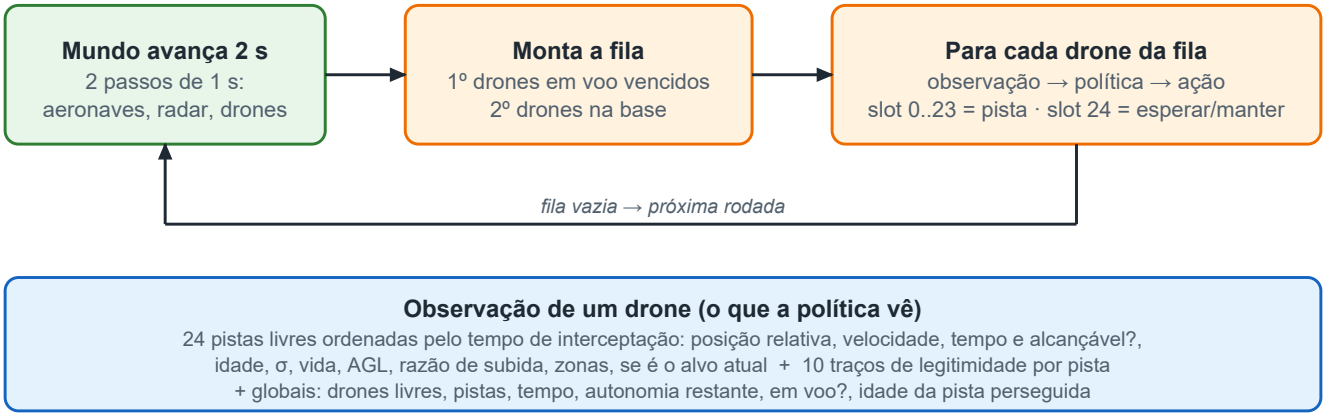


Figura 5 – Rodada de decisão e conteúdo da observação.

4.1 Recompensa

Evento	Valor
Aeronave reabastecida	+1
Queda dentro de zona prioritária	−10
Queda fora das zonas	−1
Cada passo com aeronaves ainda na lista	−(aeronaves restantes) · dt / (N · 300)

4.2 Pista legítima ou falso alarme

Antes de alocar, cada método decide se cada pista é uma aeronave. As heurísticas usam uma **regra física** (envelope de voo); as redes têm uma **sub-rede classificadora** treinada com o gabarito da simulação, mas que na decisão só usa traços de radar:

Regra física (heurísticas)	Traços da classificadora (redes)
velocidade horizontal 15–42 m/s |razão de subida| ≤ 8 m/s altura sobre o terreno −20 a 500 m	velocidade · razão de subida e seu módulo · altura sobre o terreno · σ · idade · tempo de vida · nº de medidas · medidas por segundo · distância ao radar mais próximo

5. Métodos comparados

Método	Como escolhe a pista
aleatorio	Lança para uma pista qualquer; nunca troca de alvo (piso de referência).
guloso_proximo	A pista que o drone alcança mais rápido.
guloso_urgente	A pista que parece mais perto de cair (altura ÷ razão de descida).
guloso_prioridade	Tempo de interceptação dividido pelo peso da zona prioritária.
hungaro	Atribuição ótima conjunta (algoritmo húngaro) de todos os drones da rodada.
hungaro_prioridade	Idem, ponderado pelas zonas. É a 'professora' da imitação.
hungaro_prio_sem_filtro	Idem, sem descartar pistas falsas (mede o valor do filtro).
NN_imitacao	Rede treinada por imitação (DAgger) da hungaro_prioridade.
PPO	Rede treinada do zero por reforço (PPO).
IL+PPO	Rede de imitação refinada com PPO.

Nas heurísticas, um drone em voo só troca de alvo se a nova pista for pelo menos 30 % mais rápida de alcançar (histerese), para não ficar indeciso.

5.1 Arquitetura da rede

As três redes têm a mesma arquitetura (`AllocNet`). Cada pista é processada pelo mesmo codificador e o contexto é a média dos vetores: a rede é **invariante à ordem** das pistas e funciona com qualquer número de aeronaves, drones e radares, então um único modelo serve para todas as missões.

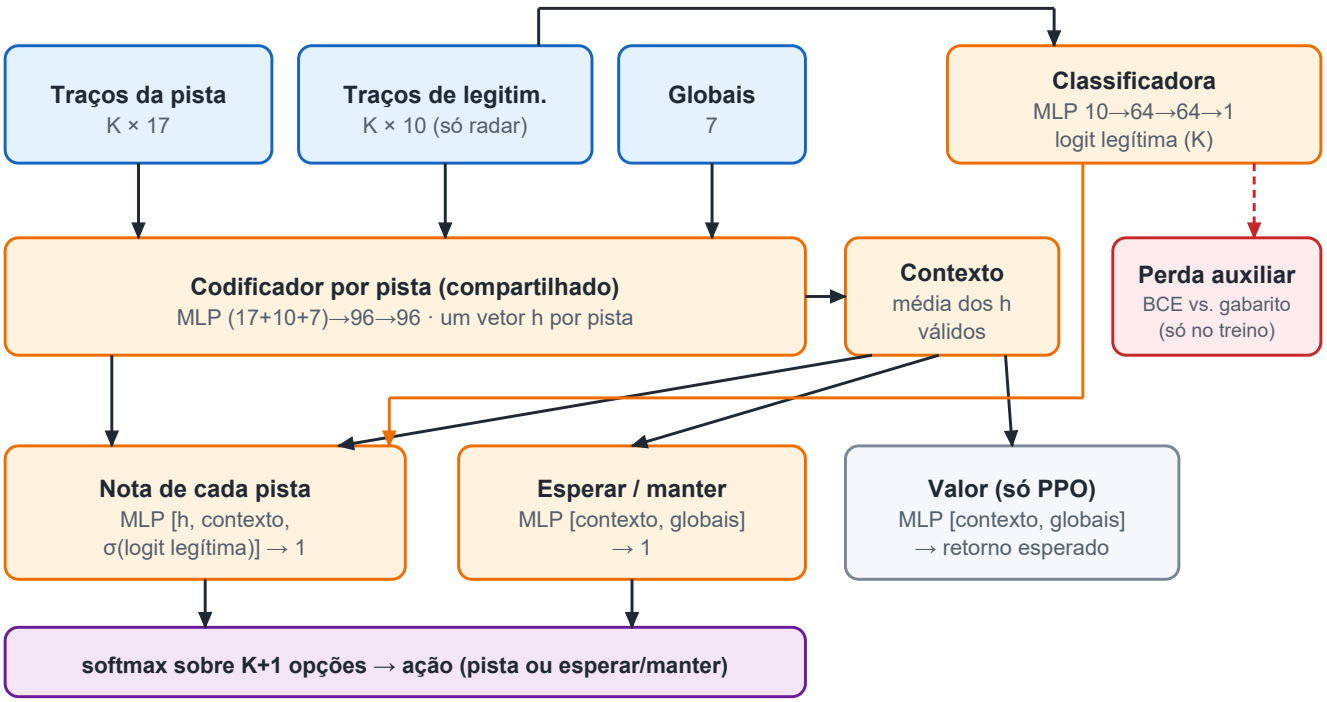


Figura 6 – AllocNet. K = 24 pistas. A probabilidade de a pista ser legítima entra na nota de cada pista; a perda auxiliar (vermelho) só existe no treino.

6. Treino e avaliação

`run_refuel.py` carrega todas as missões da pasta, separa cenários de treino e de teste, treina as três redes, avalia os dez métodos com **as mesmas sementes** (todos enfrentam exatamente os mesmos sorteios) e grava tudo numa pasta com carimbo de data e hora. Os episódios rodam em 7 processos em paralelo; o rastreador de radar é a parte mais cara de cada passo.

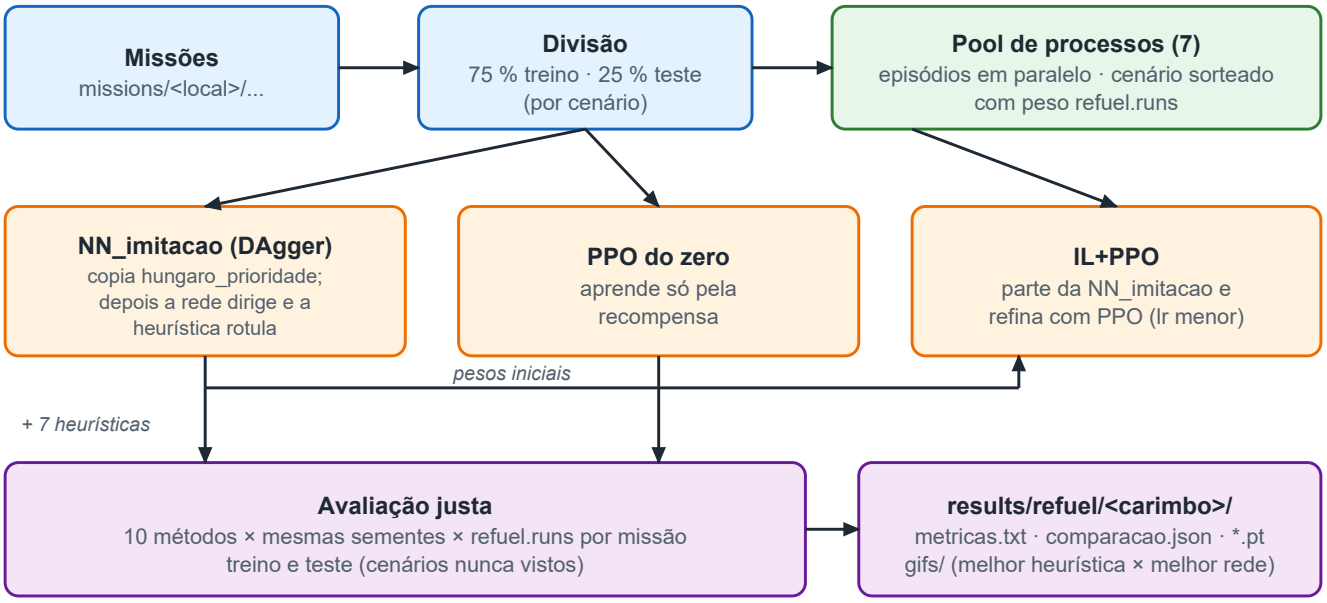


Figura 7 – Pipeline de treino e avaliação.

Arquivo de saída	Conteúdo
metricas.txt	Tabelas de treino e teste (a da Seção 7).
comparacao.json	Todas as métricas por método e por cenário, incl. precisão/recall da legitimidade.
il.pt · ppo.pt · il_ppo.pt	Pesos das três redes.
config.json · log.txt	Parâmetros da execução, divisão treino/teste e progresso do treino.
gifs/	Um episódio da melhor heurística e da melhor rede em cada cenário de teste, com JSON.

7. Resultados atuais

Execução `20261006_203304`: 19 cenários de São José dos Campos (14 de treino, 5 de teste), 10 sorteios por cenário, uma semente de treino. P/R = precisão e recall da decisão 'pista legítima'.

Método	Reabast. treino	Retorno treino	Reabast. teste	Quedas prior. teste	Retorno teste	Legít. P/R teste
aleatorio	77,9 %	−2,30	69,0 %	0,64	−5,86	–
guloso_proximo	91,8 %	−0,11	75,5 %	0,46	**−3,70**	96 / 92 %
guloso_urgente	90,8 %	−0,34	75,0 %	0,48	−3,93	96 / 92 %
guloso_prioridade	91,4 %	−0,14	75,5 %	0,46	−3,70	96 / 92 %
hungaro	**95,3 %**	0,86	75,0 %	0,48	−3,91	95 / 93 %
hungaro_prioridade	95,2 %	**0,93**	73,5 %	0,48	−3,92	95 / 93 %
hungaro_prio_sem_filtro	94,6 %	0,93	**77,0 %**	0,46	−3,71	–
NN_imitacao	94,1 %	0,60	72,0 %	0,52	−4,45	87 / 96 %
PPO	78,6 %	−2,30	67,0 %	0,62	−5,83	**98** / 93 %
IL+PPO	91,7 %	−0,30	69,5 %	0,52	−4,42	90 / **97 %**

7.1 Leitura

- **A troca de alvo em voo foi o maior ganho.** Na missão original, a heurística húngara passou de 60 % para 100 % de reabastecimento quando drones em voo puderam abandonar pistas perdidas ou falsas.
- **A rede de imitação reproduz a professora** (94 % contra 95 % no treino), mas não a supera. O IL+PPO não melhorou com o refino curto (60 atualizações).
- **O PPO do zero aprendeu o melhor classificador** de pistas (98 % de precisão), mas aloca mal e fica indeciso (52 trocas de alvo por episódio no treino).
- **Os cenários de teste são mais difíceis para todos** (~70–75 %); um deles é uma missão com 1 radar em que nenhum método reabastece.
- Para uma conclusão publicável sobre as redes faltam: mais cenários de treino, PPO mais longo, penalidade para trocas de alvo excessivas e várias sementes de treino (média e desvio).

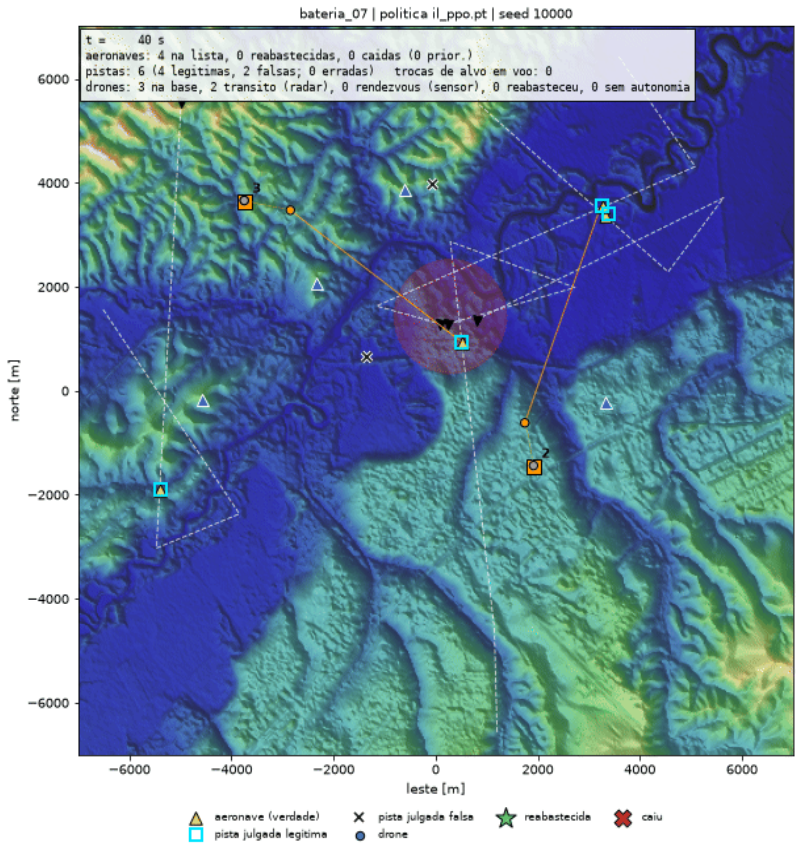


Figura 8 – Replay da rede IL+PPO na missão de teste bateria_07, t = 40 s: 4 pistas julgadas legítimas (□), 2 julgadas falsas (×), nenhum erro; dois drones em trânsito (linhas laranja).

8. Como usar

Use o Python do ambiente virtual do projeto (é onde está o PyTorch).

Criar missões

```
python mission_builder.py                       # construtor; Ctrl+S salva .json + .png
python make_battery.py --n 16 --runs 10 --seed 0  # bateria para Sao Jose dos Campos
```

Treinar e comparar

```
.venv\Scripts\python.exe run_refuel.py --missions missions/sao_jose_dos_campos \
      --il-iters 6 --ppo-updates 60            # resultados em results/refuel//
```

Ver um episódio

```
.venv\Scripts\python.exe view_refuel.py --policy hungaro_prioridade --seed 3
.venv\Scripts\python.exe view_refuel.py --policy results/refuel//il_ppo.pt
      [--save episodio.gif]   # teclas: espaço pausa, ←/→ ±10 s, R reinicia
```

Estrutura do código

Arquivo	Papel
swarm_sim/mission.py	Formato da missão, presets aleatórios, organização em pastas.
swarm_sim/missionmap.py	Mapa PNG da missão.
swarm_sim/radar.py	Radares, cobertura, rastreador IMM, quadro de pistas.
swarm_sim/refuel/env.py	Ambiente: aeronaves, drones, rendezvous, decisões, observação, legitimidade.
swarm_sim/refuel/baselines.py	Heurísticas, regra física, avaliação e métricas.
swarm_sim/refuel/learn.py	AllocNet, imitação (DAgger) e PPO.
swarm_sim/refuel/parallel.py	Episódios em processos paralelos.
swarm_sim/refuel/replay.py	Gravação e animação de episódios (janela e GIF).
make_battery.py · run_refuel.py · view_refuel.py	Scripts de linha de comando.